13

BÖLÜM: K8S İŞ YÜKLERI & DAYANIKLILIK | MODULE: K8S WORKLOADS & SELF-HEALING RESILIENCE

ReplicaSet ve Deployment: Ölümsüz Pod'lar

ReplicaSet and Deployment: Self-Healing Pods

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Tek başına bir Pod sokakta sahipsiz kalmış bir kedi yavrusu gibidir; arabaya çarparsa (çökerse) kimse arkasından ağlamaz! Deployment ise o kediye bakan 24 saat nöbetteki koruyucu melek ordusudur. 'Ben burada her zaman 3 adet sağlıklı Spring Boot kedisi istiyorum' dersiniz. Biri uyuyakalır veya ölürse, Deployment anında yenisini doğurur!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

A bare Pod is an unprotected orphan: if it crashes, nobody sheds a tear! A Deployment is its 24/7 guardian angel. You declare: 'I want exactly 3 healthy Spring Boot pods at all times.' If one pod gets killed or its server explodes, the Deployment controller instantly breathes life into an identical replacement!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Üretim ortamında neredeyse HİÇBİR ZAMAN çıplak Pod (bare pod) oluşturulmaz; her zaman Deployment kullanılır. Deployment hiyerarşisi şöyledir: Deployment -> ReplicaSet -> Pods. Deployment, yeni sürümler çıktığında yeni bir ReplicaSet açarak eski ReplicaSet'teki pod'ları kademeli olarak azaltır (Rolling Update). `spec.replicas` sayısı etcd'deki Desired State'tir. Pod çöktüğünde ReplicaSet controller bunu anında fark edip yeni bir Pod talebi oluşturur.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Bare pods are strictly anti-pattern in production; Deployments are mandatory. The architecture follows a strict hierarchy: Deployment -> ReplicaSet -> Pods. The Deployment controller manages declarative updates by spinning up a new ReplicaSet while scaling down the old one. If a container crashes, the Kubelet or ReplicaSet reconciles Pod counts back to the declared replica integer.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```yaml
# spring-boot-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: coffee-deployment
  labels:
    app: coffee-app
spec:
  replicas: 3 # 3 adet yedekli pod çalıştır
  selector:
    matchLabels:
      app: coffee-app
  template:
    metadata:
      labels:
        app: coffee-app
    spec:
      containers:
        - name: coffee-container
          image: ghcr.io/eli5-k8s/spring-boot-k8s-eli5:1.0.0
          ports:
            - containerPort: 8080
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Deployment'ın `matchLabels` ile Pod şablonundaki (`template.metadata.labels`) etiketler birebir AYNI olmalıdır! Uyuşmazsa Deployment oluşturduğu pod'ları tanıyamaz ve sonsuz döngüde pod basmaya başlar.

🇬🇧 WATCH OUT

The `spec.selector.matchLabels` MUST match `spec.template.metadata.labels` identically. A mismatch causes the controller to lose track of its pods and trigger endless creation loops.

🎯 Özet Çıkarımlar

🎯 Key Takeaways

- Deployment, pod'ların çökmesi durumunda anında kendi kendini iyileştirir.
- Arka planda ReplicaSet nesneleri sürüm geçmişini ve kopya sayısını yönetir.
- Tek bir komutla (`kubectl scale deployment ... --replicas=5`) anında ölçeklenir.

- Deployments guarantee self-healing: dead pods are instantly replaced.
- ReplicaSets handle the low-level replica counting and historical revisions.
- Scale your fleet on demand with `kubectl scale deployment --replicas=N`.